

Contents

1 LTIC systems and time domain, systemklassifikation, tidsdomæne	1
1.1 Multiple-choice strategy, one best answer, sande falske udsagn	1
1.2 Signals and elementary functions, signaler, impuls, step, rampe	1
1.3 System classification, lineær, tidsinvariant, kausal, stabil	1
1.4 Differential equations and transfer functions, differentiaaligning, overføringsfunktion	2
1.5 Circuit equations, knudepunktsgligning, kredsløb	2
1.6 Impulse, step, ramp response, impulsrespons, steprespons, ramperespons	2
1.7 Convolution, foldning, tabelopslag	3
1.8 Second-order systems, anden orden, damping, Q	3
2 Fourier series and Fourier transform, Fourierrække, Fourier-transformation	4
2.1 Fourier series, Fourierrække, periodisk signal	4
2.2 Fourier transform, transformation, spectrum	5
2.3 Exam recipe: Fourier statement checking	5
3 Laplace transform, unilateral Laplace, ensidig Laplace	6
3.1 Definitions and regions of convergence, konvergensområde, ROC	6
3.2 Derivative rules and initial conditions, begyndelsesbetingelser	6
3.3 Zero-input and zero-state, egenrespons, påtvungen respons	6
3.4 Inverse Laplace and partial fractions, invers Laplace, partialbrøk	6
3.5 Value theorems, beginning and final value, begyndelsesværdi, slutværdi	7
3.6 Exam recipe: Laplace response task	7
4 Frequency response, Bode, pole-zero diagrams, filters	7
4.1 Frequency response, frekvenskarakteristik	7
4.2 Bode plot rules, Bodeplot, asymptoter	8
4.3 Filter classification, lavpas, højpas, båndpas, båndstop	8
4.4 Second-order resonance and system identification	8
4.5 Butterworth filters, Butterworth filter, Sallen-Key	9
5 Sampling, ADC, instrumentation amplifier, instrumenteringsforstærker	10
5.1 Sampling and aliasing, sampling, aliasering, Nyquist	10
5.2 ADC quantization, LSB, kvantisering, dynamic range	10
5.3 Anti-alias filter sizing, ADC and Butterworth	10
5.4 Instrumentation amplifier, differential gain, common mode, CMRR	11
5.5 Exam recipe: ADC and instrumentation MCQ checks	11
6 Validated Python aids, Python-funktioner, kode	11
6.1 How to use scripts during the exam	11

1 LTIC systems and time domain, systemklassifikation, tidsdomæne

1.1 Multiple-choice strategy, one best answer, sande falske udsagn

1.1.1 One-best-answer elimination

Each answer option can contain several claims. If one claim in an option is false, the whole option is false.

Fast checks: test limiting cases, signs, pole locations, units, and whether a statement is universal or only true under extra assumptions. If more than one option contains only true statements, the exam may use the special “more than one” answer.

1.2 Signals and elementary functions, signaler, impuls, step, rampe

1.2.1 Unit impulse, Dirac impulse, enhedsimpuls

$$\int_{-\infty}^{\infty} \delta(t - t_0) dt = 1, \quad x(t)\delta(t - t_0) = x(t_0)\delta(t - t_0)$$

The impulse has area, not ordinary amplitude. Multiplication by a smooth signal samples the signal value at the impulse location.

1.2.2 Unit step and ramp, enhedstrin, stepfunktion, rampe

$$u(t) = \begin{cases} 0, & t < 0 \\ 1, & t > 0 \end{cases}, \quad r(t) = t u(t)$$
$$\frac{d}{dt}u(t) = \delta(t), \quad \frac{d}{dt}r(t) = u(t)$$

Use these relations to move between impulse, step, and ramp responses. The value exactly at $t = 0$ is normally not decisive for these exam tasks.

1.3 System classification, lineær, tidsinvariant, kausal, stabil

1.3.1 Linearity of systems

$$x_1 \mapsto y_1, \quad x_2 \mapsto y_2 \quad \Rightarrow \quad ax_1 + bx_2 \mapsto ay_1 + by_2$$

A differential equation with real constant coefficients and only linear combinations of x, y and their derivatives is linear. Terms such as $x^2(t)$, $y^2(t)$, or products of signals break linearity.

1.3.2 Time invariance, tidsinvarians

$$x(t) \mapsto y(t) \quad \Rightarrow \quad x(t - t_0) \mapsto y(t - t_0)$$

Constant coefficients support time invariance. A coefficient such as $t y(t)$ or a component value that changes with time breaks time invariance.

1.3.3 Causality, kausalitet

A system is causal when $y(t_0)$ depends only on $x(t)$ for $t \leq t_0$. A term such as $x(t + 2)$ is non-causal because it uses future input.

Fast check: physical passive/active circuits in this course are causal unless the mathematical expression explicitly uses future input.

1.3.4 BIBO stability and pole locations, stabilitet

$$\text{stable causal rational LTIC} \quad \Longleftrightarrow \quad \operatorname{Re}\{p_k\} < 0 \text{ for every pole } p_k$$

Positive real-part poles give growing natural modes and instability. Poles on the imaginary axis are not asymptotically stable and require special interpretation.

1.4 Differential equations and transfer functions, differentiaaligning, overføringsfunktion

1.4.1 Standard LTIC differential equation

$$\sum_{k=0}^N a_k \frac{d^k y}{dt^k} = \sum_{m=0}^M b_m \frac{d^m x}{dt^m}$$

With zero initial conditions,

$$H(s) = \frac{Y(s)}{X(s)} = \frac{\sum_{m=0}^M b_m s^m}{\sum_{k=0}^N a_k s^k}.$$

Use exam notation $x(t)$ for input and $y(t)$ for output unless the problem states otherwise.

Python aid: Use `scripts/lti/responses.py` function `transfer_from_ode` when the problem gives numerator and denominator coefficient lists and asks for a quick symbolic transfer function, poles, or zeros. Inputs are coefficient lists in descending powers of s . The script returns $H(s)$, zeros, and poles. Check manually that initial conditions are zero before interpreting it as a transfer function.

1.4.2 Filter type from numerator powers

$$\frac{b_0}{s^2 + a_1s + a_0} \text{ lowpass, } \quad \frac{b_1s}{s^2 + a_1s + a_0} \text{ bandpass, } \quad \frac{b_2s^2}{s^2 + a_1s + a_0} \text{ highpass}$$

This classification assumes a stable proper second-order denominator. Always confirm by checking the limits $s \rightarrow 0$ and $|s| \rightarrow \infty$.

1.5 Circuit equations, knudepunktsligning, kredsløb

1.5.1 Component laws in time domain

$$i_R = \frac{v_R}{R}, \quad i_C = C \frac{dv_C}{dt}, \quad v_L = L \frac{di_L}{dt}$$

Write KCL at each unknown node with a consistent current direction. Capacitor voltage cannot jump unless an impulse current is present; inductor current cannot jump unless an impulse voltage is present.

1.5.2 Component laws in phasor domain, vekselstrømskredsløb

$$Z_R = R, \quad Z_C = \frac{1}{j\omega C}, \quad Z_L = j\omega L$$

Use impedance algebra to find $H(j\omega) = Y(j\omega)/X(j\omega)$. For op-amp circuits, apply ideal input current 0 and virtual short only when negative feedback makes the ideal assumption valid.

1.5.3 Circuit limiting checks, DC and high frequency

$$\omega \rightarrow 0 : C \text{ open, } L \text{ short; } \quad \omega \rightarrow \infty : C \text{ short, } L \text{ open}$$

These checks often identify lowpass, highpass, and bandpass behavior before full algebra. They are also useful for eliminating false MCQ options.

1.6 Impulse, step, ramp response, impulsrespons, steprespons, ramperespons

1.6.1 Convolution input-output relation

$$y_{zs}(t) = h(t) * x(t) = \int_{-\infty}^{\infty} h(\tau)x(t-\tau) d\tau$$

For an LTIC system, the impulse response fully determines the zero-state response. Zero-input response is caused by initial conditions and is not computed from convolution with the input.

1.6.2 Impulse response from transfer function

$$h(t) = \mathcal{L}^{-1}\{H(s)\}$$

If $H(s)$ is improper or has equal numerator and denominator degree, $h(t)$ may contain direct impulse terms such as $K\delta(t)$.

Python aid: Use `scripts/lti/responses.py` function `impulse_response_from_transfer` when the problem asks for an impulse response from a rational transfer function and the required inputs are numerator and denominator coefficients in descending powers of s . The script returns a symbolic expression for $h(t)$. Check manually that the coefficient order is correct and whether a direct $\delta(t)$ term is expected. Do not use it for conceptual statements about causality or zero-input response.

1.6.3 Step and ramp response relations

$$y_{\text{step}}(t) = h(t) * u(t), \quad h(t) = \frac{d}{dt}y_{\text{step}}(t)$$
$$y_{\text{ramp}}(t) = y_{\text{step}}(t) * u(t) = \int_0^t y_{\text{step}}(\tau) d\tau$$

Use differentiation to go from step response to impulse response and integration to go from step response to ramp response. Watch for impulse terms when the step response has a jump at 0^+ .

Python aid: Use `scripts/lti/responses.py` function `step_response_from_transfer` when the problem asks for a zero-state unit-step response and the required inputs are numerator and denominator coefficients of $H(s)$. Use `ramp_response_from_transfer` for ramp response from $H(s)$. Use `related_unit_responses` when one of $h(t)$, $y_{\text{step}}(t)$, or $y_{\text{ramp}}(t)$ is known and the other two are needed. Check manually that initial conditions are zero and that the task is not asking for total response with stored energy.

1.7 Convolution, foldning, tabelopslag

1.7.1 Convolution integral

$$(x_1 * x_2)(t) = \int_{-\infty}^{\infty} x_1(\tau)x_2(t - \tau) d\tau$$

Convolution is commutative and associative. In LTIC systems it computes zero-state response from input and impulse response.

1.7.2 Support interval rule, tidsinterval

$$x_1(t) = 0 \text{ outside } (t_1, t_2), \quad x_2(t) = 0 \text{ outside } (t_3, t_4) \\ x_1 * x_2 = 0 \text{ outside } (t_1 + t_3, t_2 + t_4)$$

This rule answers support questions without calculating the full convolution.

1.7.3 Causal exponential convolution

$$\mathcal{L}\{x_1 * x_2\} = X_1(s)X_2(s)$$

For causal exponential signals, Laplace multiplication and inverse transformation are usually faster than direct integration.

Python aid: Use `scripts/transforms/convolution.py` function `convolve_causal` when the problem asks for explicit convolution of causal signals such as $e^{-at}u(t)$ or $te^{-at}u(t)$. Inputs are the two SymPy expressions without the final $u(t)$. The script returns the symbolic causal convolution. Check manually that both signals are causal and that the problem is not only asking for the support interval.

1.8 Second-order systems, anden orden, damping, Q

1.8.1 Standard second-order denominator

$$s^2 + a_1s + a_0 = s^2 + 2\zeta\omega_n s + \omega_n^2 \\ \omega_n = \sqrt{a_0}, \quad \zeta = \frac{a_1}{2\sqrt{a_0}}, \quad Q = \frac{1}{2\zeta}$$

Use this mapping for second-order ODEs and transfer functions with denominator $s^2 + a_1s + a_0$.

1.8.2 Pole locations and damping class

$$s_{1,2} = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}$$

Condition	Pole type	Exam word
$\zeta > 1$	two real negative poles	overdamped, overdæmpet
$\zeta = 1$	repeated real pole	critically damped
$0 < \zeta < 1$	complex-conjugate poles	underdamped, underdæmpet
$\zeta < 0$	growing modes	unstable, ustabil

Python aid: Use `scripts/lti/second_order.py` function `classify_second_order` when the problem gives $s^2 + a_1s + a_0$ and asks for poles, damping, stability, Q , or ω_n . Inputs are a_1, a_0 . The script returns poles, ζ , ω_n , Q , stability, and damping class. Check manually that the denominator is normalized with leading coefficient 1.

1.8.3 Underdamped step features, overshoot, peak time

$$PO = 100e^{-\zeta\pi/\sqrt{1-\zeta^2}}, \quad t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}}, \quad t_s \approx \frac{4}{\zeta\omega_n} \\ \zeta = \frac{-\ln(PO/100)}{\sqrt{\pi^2 + \ln^2(PO/100)}}, \quad \omega_d = \frac{\pi}{t_p}, \quad \omega_n = \frac{\omega_d}{\sqrt{1-\zeta^2}}$$

Use these only for $0 < \zeta < 1$. $t_s \approx 4/(\zeta\omega_n)$ is the usual 2 percent settling-time approximation.

Python aid: Use `scripts/lti/second_order.py` function `step_features_from_zeta_wn` when ζ and ω_n are already known and the problem asks for overshoot, peak time, damped frequency, or settling time. Inputs are ζ, ω_n . The script returns PO, t_p, ω_d, t_s . Check manually that $0 < \zeta < 1$.

Python aid: Use `scripts/lti/second_order.py` function `estimate_from_overshoot_peak_time` when a step plot gives percent overshoot and first peak time. Inputs are PO in percent and t_p . The script returns $\zeta, \omega_n, \omega_d, t_s$. Check manually that the plotted response is underdamped and second order. Do not use it for overdamped responses.

1.8.4 Sensitivity of underdamped pole coordinates

$$\sigma = -\zeta\omega_n, \quad \omega_d = \omega_n\sqrt{1-\zeta^2}, \quad S_x^y = \frac{x}{y} \frac{\partial y}{\partial x}$$

$$S_\zeta^\sigma = 1, \quad S_{\omega_n}^\sigma = 1, \quad S_\zeta^{\omega_d} = -\frac{\zeta^2}{1-\zeta^2}, \quad S_{\omega_n}^{\omega_d} = 1$$

As $\zeta \rightarrow 1^-$, $\omega_d \rightarrow 0$ and $S_\zeta^{\omega_d} \rightarrow -\infty$. This is a recurring sensitivity MCQ trap: the negative sign in $\sigma = -\zeta\omega_n$ does not make S_ζ^σ negative.

1.8.5 Second-order lowpass, bandpass, highpass gains

$$H_{LP}(s) = K \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

$$H_{BP}(s) = K \frac{2\zeta\omega_n s}{s^2 + 2\zeta\omega_n s + \omega_n^2}, \quad H_{HP}(s) = K \frac{s^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

For a lowpass, DC gain is K . For a highpass, high-frequency gain is K . A bandpass is zero at DC and at infinite frequency.

2 Fourier series and Fourier transform, Fourierrække, Fourier-transformation

2.1 Fourier series, Fourierrække, periodisk signal

2.1.1 Complex exponential Fourier series

$$x(t) = \sum_{n=-\infty}^{\infty} D_n e^{jn\omega_0 t}, \quad D_n = \frac{1}{T_0} \int_{t_1}^{t_1+T_0} x(t) e^{-jn\omega_0 t} dt$$

$$\omega_0 = \frac{2\pi}{T_0}$$

The complex exponentials are orthogonal over one period. Expanding a finite time window of an aperiodic signal creates a periodic reconstruction with period T_0 .

Python aid: Use `scripts/fourier/series.py` function `complex_fourier_coefficients` when the problem asks for symbolic D_n coefficients over a known interval. Inputs are the SymPy expression, time symbol, period, requested integer indices, and interval start. The script returns a dictionary of coefficients. Check manually that the chosen interval is one full period and that the signal is integrable over it.

2.1.2 Trigonometric Fourier series

$$x(t) = a_0 + \sum_{n=1}^{\infty} (a_n \cos(n\omega_0 t) + b_n \sin(n\omega_0 t))$$

$$a_n = \frac{2}{T_0} \int_{t_1}^{t_1+T_0} x(t) \cos(n\omega_0 t) dt, \quad b_n = \frac{2}{T_0} \int_{t_1}^{t_1+T_0} x(t) \sin(n\omega_0 t) dt$$

Use cosine terms for even parts and sine terms for odd parts. Confirm the convention for a_0 if comparing with a source using $a_0/2$.

2.1.3 Fourier series symmetry, lige ulige

$$x(t) \text{ real} \Rightarrow D_{-n} = D_n^*$$

$$x(t) \text{ real and even} \Rightarrow D_n = D_{-n} \in \mathbb{R}$$

$$x(t) \text{ real and odd} \Rightarrow D_n = -D_{-n} \text{ and } D_n \text{ is imaginary}$$

Magnitude $|D_n|$ cannot be negative. This is a common false-statement trap.

2.2 Fourier transform, transformation, spectrum

2.2.1 Fourier transform definition

$$X(\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt, \quad x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(\omega) e^{j\omega t} d\omega$$

The exam uses angular frequency ω in rad/s. Some periodic signals are handled with generalized transforms, so avoid claiming that Fourier transforms are only for aperiodic signals.

2.2.2 Basic transform pairs

$$\begin{aligned}\delta(t) &\leftrightarrow 1, & u(t)e^{-at} &\leftrightarrow \frac{1}{a + j\omega}, & a > 0 \\ \cos(\omega_0 t) &\leftrightarrow \pi [\delta(\omega - \omega_0) + \delta(\omega + \omega_0)] \\ \sin(\omega_0 t) &\leftrightarrow \frac{\pi}{j} [\delta(\omega - \omega_0) - \delta(\omega + \omega_0)]\end{aligned}$$

For $e^{-at}u(t)$, the sign of a determines convergence. The transform exists as an ordinary decaying signal transform for $a > 0$.

2.2.3 Fourier transform properties, skalering, modulation, tidsforskydning

$$\begin{aligned}x(t - t_0) &\leftrightarrow e^{-j\omega t_0} X(\omega) \\ x(at) &\leftrightarrow \frac{1}{|a|} X\left(\frac{\omega}{a}\right), & x(t)e^{j\omega_c t} &\leftrightarrow X(\omega - \omega_c) \\ \frac{dx}{dt} &\leftrightarrow j\omega X(\omega), & x_1(t)x_2(t) &\leftrightarrow \frac{1}{2\pi} X_1(\omega) * X_2(\omega)\end{aligned}$$

Time compression broadens the spectrum. Time shift changes phase but not magnitude.

Python aid: Use `scripts/fourier/properties.py` function `exponential_transform_variant` when the problem starts from $e^{-at}u(t)$ and asks for a scaled or modulated transform pair. Inputs are a , positive time scale, and modulation frequency. The script returns the transformed expression. Check manually that the operation is scaling or modulation, not time shifting or a conceptual existence question.

2.2.4 Even and odd transform symmetry

$$x(t) = x_e(t) + x_o(t), \quad x_e(t) = \frac{x(t) + x(-t)}{2}, \quad x_o(t) = \frac{x(t) - x(-t)}{2}$$

For real even $x(t)$, $X(\omega)$ is real and even. For real odd $x(t)$, $X(\omega)$ is imaginary and odd.

2.3 Exam recipe: Fourier statement checking

2.3.1 Fourier MCQ checklist

Use when: an answer option contains several claims about Fourier series, Fourier transforms, symmetry, or transform properties.

Steps:

1. Identify whether the task is series D_n or transform $X(\omega)$.
2. Check signal symmetry before integrating.
3. Apply only one property at a time: scaling, modulation, time shift, or differentiation.
4. Check units: f in Hz and $\omega = 2\pi f$ in rad/s are not interchangeable.

Fast checks: real even signals have zero imaginary transform part; real odd signals have zero real transform part. Scaling by a must include $1/|a|$.

Common traps: Fourier transform is not restricted to only aperiodic signals; $|D_n|$ is never negative; time shift does not change magnitude spectrum.

3 Laplace transform, unilateral Laplace, ensidig Laplace

3.1 Definitions and regions of convergence, konvergensområde, ROC

3.1.1 Unilateral Laplace transform

$$X(s) = \int_{0^-}^{\infty} x(t)e^{-st} dt$$

The exams state that Laplace transforms are unilateral unless otherwise specified. The lower limit 0^- includes impulses or initial-condition effects at the switching instant.

3.1.2 Causal exponential and ROC

$$e^{at}u(t) \leftrightarrow \frac{1}{s - a}, \quad \text{Re}\{s\} > a$$

For a causal signal, the ROC lies to the right of the rightmost pole. For a stable causal system, the imaginary axis must be inside the ROC.

3.2 Derivative rules and initial conditions, begyndelsesbetingelser

3.2.1 Unilateral derivative rules

$$\begin{aligned}\mathcal{L}\{\dot{y}(t)\} &= sY(s) - y(0^-) \\ \mathcal{L}\{\ddot{y}(t)\} &= s^2Y(s) - sy(0^-) - \dot{y}(0^-)\end{aligned}$$

These terms are why unilateral Laplace can include initial conditions. Use the values at 0^- unless the problem explicitly gives 0^+ .

3.2.2 Transfer function from differential equation

$$\begin{aligned}y'' + a_1y' + a_0y &= b_2x'' + b_1x' + b_0x \\ H(s) = \frac{Y(s)}{X(s)} &= \frac{b_2s^2 + b_1s + b_0}{s^2 + a_1s + a_0} \quad \text{when all initial conditions are zero.}\end{aligned}$$

If initial conditions are nonzero, $Y(s)$ contains both $H(s)X(s)$ and a zero-input term.

3.3 Zero-input and zero-state, egenrespons, påtvungen respons

3.3.1 Response decomposition

$$y(t) = y_{zi}(t) + y_{zs}(t), \quad Y(s) = Y_{zi}(s) + H(s)X(s)$$

Zero-input response comes from stored energy or initial conditions. Zero-state response comes from the input with zero initial conditions.

3.3.2 Second-order zero-input Laplace expression

For

$$\begin{aligned}y'' + a_1y' + a_0y &= 0, \quad y(0^-) = y_0, \quad \dot{y}(0^-) = v_0, \\ Y_{zi}(s) &= \frac{sy_0 + v_0 + a_1y_0}{s^2 + a_1s + a_0}.\end{aligned}$$

This formula appears directly in several zero-input MCQ options.

Python aid: Use `scripts/lti/responses.py` function `zero_input_laplace_second_order` when the problem gives $a_1, a_0, y(0^-), \dot{y}(0^-)$ and asks for $Y_{zi}(s)$. The script returns the rational Laplace expression. Check manually that the ODE is normalized to $y'' + a_1y' + a_0y = 0$. Do not use it for higher-order systems.

3.4 Inverse Laplace and partial fractions, invers Laplace, partialbrøk

3.4.1 Common inverse pairs

$$\begin{aligned}\frac{1}{s+a} &\leftrightarrow e^{-at}u(t), & \frac{1}{(s+a)^2} &\leftrightarrow te^{-at}u(t) \\ \frac{\omega}{(s+a)^2 + \omega^2} &\leftrightarrow e^{-at}\sin(\omega t)u(t) \\ \frac{s+a}{(s+a)^2 + \omega^2} &\leftrightarrow e^{-at}\cos(\omega t)u(t)\end{aligned}$$

Complete the square when the denominator has complex-conjugate poles.

3.4.2 Partial fraction decomposition

$$\frac{1}{s(s+a)} = \frac{1}{a} \left(\frac{1}{s} - \frac{1}{s+a} \right)$$

Repeated poles produce polynomial factors in time. Always simplify before comparing to MCQ options because equivalent expressions may be written differently.

Python aid: Use `scripts/transforms/laplace.py` function `inverse_laplace_rational` when the problem asks for an inverse Laplace transform of a rational expression and the numerator and denominator coefficients are clear. The script returns a symbolic time-domain expression. Check manually for repeated poles, direct terms, and whether the problem requires a proof or only a final expression.

3.5 Value theorems, beginning and final value, begyndelsesværdi, slutværdi

3.5.1 Initial value theorem

$$y(0^+) = \lim_{s \rightarrow \infty} sY(s)$$

Use this only when the limit exists. It is useful for checking whether a step response has an initial jump.

3.5.2 Final value theorem

$$\lim_{t \rightarrow \infty} y(t) = \lim_{s \rightarrow 0} sY(s)$$

The theorem is valid only if every pole of $sY(s)$ is in the open left half-plane except possibly a simple pole at the origin.

Common trap: applying the final value theorem to an unstable or oscillatory response gives false conclusions.

3.6 Exam recipe: Laplace response task

3.6.1 Laplace response workflow

Use when: an LTIC task asks for transfer function, impulse response, step response, zero-input response, zero-state response, or final value.

Steps:

1. Convert the differential equation using unilateral derivative rules.
2. Separate initial-condition terms from $H(s)X(s)$.
3. Use partial fractions for inverse Laplace.
4. Check poles before using the final value theorem.

Fast checks: denominator roots are natural modes; zero-state with $x(t) = u(t)$ uses $X(s) = 1/s$; a highpass step response should settle to zero.

4 Frequency response, Bode, pole-zero diagrams, filters

4.1 Frequency response, frekvenskarakteristik

4.1.1 Transfer function on the imaginary axis

$$H(j\omega) = H(s)\Big|_{s=j\omega}, \quad |H|_{\text{dB}} = 20 \log_{10} |H|, \quad \angle H = \arg H$$

Use $H(j\omega)$, not $H(\omega)$, when substituting into a transfer function. The complex unit is j in the course notation.

Python aid: Use `scripts/frequency/bode.py` function `bode_values` when the problem asks for numerical magnitude or phase checks at selected angular frequencies. Inputs are numerator coefficients, denominator coefficients, and ω values. The script returns $H(j\omega)$, magnitude in dB, and phase in degrees. Check manually that the frequency axis is in rad/s and that the coefficients are ordered by descending powers of s .

4.1.2 Magnitude and phase from factors

$$H(j\omega) = K \frac{\prod_i (j\omega - z_i)}{\prod_k (j\omega - p_k)}$$
$$|H| = |K| \frac{\prod_i |j\omega - z_i|}{\prod_k |j\omega - p_k|}, \quad \angle H = \angle K + \sum_i \angle(j\omega - z_i) - \sum_k \angle(j\omega - p_k)$$

Magnitude is a distance ratio in the pole-zero diagram. Phase is an angle sum from zeros minus poles.

4.2 Bode plot rules, Bodeplot, asymptoter

4.2.1 First-order Bode asymptotes

$$1 + \frac{s}{\omega_c}$$

A zero adds +20 dB/decade after ω_c and positive phase. A pole adds -20 dB/decade after ω_c and negative phase.

4.2.2 Origin poles and zeros

$$\begin{aligned} s^m &\Rightarrow +20m \text{ dB/decade}, & +90m^\circ \\ \frac{1}{s^m} &\Rightarrow -20m \text{ dB/decade}, & -90m^\circ \end{aligned}$$

Repeated poles or zeros multiply the slope and phase contribution.

4.2.3 Bode matching recipe

Use when: an exam task asks which transfer function matches a Bode plot.

Steps:

1. Read low-frequency slope to count poles or zeros at the origin.
2. Read break frequencies and slope changes.
3. Check high-frequency slope from numerator degree minus denominator degree.
4. Use phase start and end as a sign check.

Fast checks: each real pole changes phase by about -90° ; each real zero changes phase by about $+90^\circ$. A second-order pair changes slope by 40 dB/decade.

4.3 Filter classification, lavpas, højpas, båndpas, båndstop

4.3.1 Limit tests for filter type

$$H(0) = \lim_{s \rightarrow 0} H(s), \quad H(\infty) = \lim_{|s| \rightarrow \infty} H(s)$$

Type	$H(0)$	$H(\infty)$
Lowpass, lavpas	nonzero	zero
Highpass, højpas	zero	nonzero
Bandpass, båndpas	zero	zero
Bandstop/notch, båndstop	nonzero	nonzero with notch

Python aid: Use `scripts/frequency/bode.py` function `classify_filter_limits` when the problem gives a rational transfer function and asks for a quick lowpass/highpass/bandpass limit check. Inputs are numerator and denominator coefficients. The script returns DC gain, high-frequency gain, polynomial degrees, and a probable class. Check manually for notches, resonant peaks, and whether the system is stable.

4.3.2 Pole-zero filter interpretation

$$H(s) = K \frac{\prod_i (s - z_i)}{\prod_k (s - p_k)}$$

Zeros near the imaginary axis create attenuation dips. Poles near the imaginary axis create peaks. A zero at $s = 0$ blocks DC and pushes the response toward highpass behavior.

Phase trap: mirrored left/right half-plane zeros have the same distance to the imaginary axis and can give the same magnitude contribution, but their phase contribution differs. A right-half-plane zero is non-minimum phase.

Python aid: Use `scripts/frequency/bode.py` function `transfer_from_poles_zeros` when a pole-zero diagram gives numeric poles, zeros, and gain and the task asks for a transfer function or Bode check. The script returns numerator and denominator coefficient arrays. Check manually that complex roots occur in conjugate pairs for real systems.

4.4 Second-order resonance and system identification

4.4.1 Damped natural and resonant frequencies

$$\omega_d = \omega_n \sqrt{1 - \zeta^2}$$

$$\omega_r = \omega_n \sqrt{1 - 2\zeta^2} \quad \text{for a 2nd-order lowpass when } 0 < \zeta < \frac{1}{\sqrt{2}}$$

The damped frequency ω_d appears in time-domain oscillation. The resonant peak frequency ω_r appears in the magnitude response.

4.4.2 Step plot to Bode consistency check

Use PO and t_p from a step plot to estimate ζ and ω_n . Then compare the predicted resonant peak and phase trend with the Bode plot.

Fast checks: $t_p = \pi/\omega_d$; larger overshoot means smaller ζ ; underdamped poles must be complex conjugates.

4.5 Butterworth filters, Butterworth filter, Sallen-Key

4.5.1 Butterworth magnitude definition

$$|H(j\omega)| = \frac{1}{\sqrt{1 + \epsilon^2 (\omega/\omega_p)^{2n}}}$$

For the common 3 dB cutoff case, $\epsilon = 1$ and $\omega_p = \omega_c$. Butterworth means maximally flat magnitude, not zero phase distortion.

4.5.2 Butterworth order inequality

$$n \geq \frac{\log_{10}((10^{A_s/10} - 1)/(10^{A_p/10} - 1))}{2 \log_{10}(\omega_s/\omega_p)}$$

Use the ceiling of the computed value. For highpass design, transform the frequency ratio so the stopband frequency maps below the passband frequency.

Python aid: Use `scripts/filters/butterworth.py` function `butterworth_lowpass_order` when the problem asks for minimum lowpass Butterworth order from passband and stopband specifications. Inputs are f_p, f_s, A_p, A_s . The script returns the minimum integer order. Check manually that $f_s > f_p$ and that attenuation values are positive dB numbers. Use `butterworth_highpass_order` only when $f_s < f_p$.

4.5.3 Butterworth poles and Q values

$$s_k = \omega_c e^{j\left(\frac{\pi}{2} + \frac{(2k+1)\pi}{2n}\right)}, \quad k = 0, \dots, n-1$$

Use only the left-half-plane poles for the causal stable filter. Pair complex-conjugate poles into second-order sections; a real pole becomes a first-order section.

The squared Butterworth magnitude can be written as a product $H(s)H(-s)$. $H(s)$ is the causal stable filter using left-half-plane poles; $H(-s)$ is the anti-causal mirror with right-half-plane poles. The zero phase of the product does not mean the causal Butterworth filter has zero phase.

Python aid: Use `scripts/filters/butterworth.py` function `butterworth_poles_q` when the problem asks for Butterworth poles or second-order section Q values. Inputs are order and cutoff. The script returns stable poles and Q values. Check manually how stages are assigned to the actual Sallen-Key circuit.

4.5.4 Sallen-Key sensitivity and component matching

$$S_x^y = \frac{\partial y/y}{\partial x/x} = \frac{x}{y} \frac{\partial y}{\partial x}$$

Sensitivity measures percentage change in a dependent quantity caused by a percentage component change. In Sallen-Key design tasks, equal component choices or specific capacitor ratios are used to reduce Q -sensitivity.

Course-specific matching rule: for the unity-gain Sallen-Key lowpass in L12, matching $R_1 = R_2$ makes Q insensitive to resistor drift. For the unity-gain Sallen-Key highpass in L13, matching $C_1 = C_2$ gives low Q -sensitivity. Do not transfer the lowpass conclusion directly to the highpass topology.

4.5.5 Frequency scaling of RC filters

$$\hat{\omega}_c = 1 \text{ rad/s} \rightarrow \omega_c$$

Keeping R fixed gives $C_{\text{new}} = \hat{C}/\omega_c$. Keeping C fixed gives $R_{\text{new}} = \hat{R}/\omega_c$. Do not multiply both R and C by ω_c .

Python aid: Use `scripts/filters/butterworth.py` function `frequency_scale_rc` when a normalized RC design must be scaled to a target cutoff. Inputs are normalized R, C, ω_c and mode `keep_R` or `keep_C`. The script returns scaled component values. Check manually that the target uses rad/s; convert from Hz using $\omega_c = 2\pi f_c$.

5 Sampling, ADC, instrumentation amplifier, instrumenteringsforstærker

5.1 Sampling and aliasing, sampling, aliasing, Nyquist

5.1.1 Impulse-train sampling spectrum

$$x_s(t) = x(t) \sum_{n=-\infty}^{\infty} \delta(t - nT)$$
$$X_s(\omega) = \frac{1}{T} \sum_{n=-\infty}^{\infty} X(\omega - n\omega_s), \quad \omega_s = \frac{2\pi}{T} = 2\pi F_s$$

Sampling repeats the spectrum every ω_s . The repeated spectra are continuous shifted copies, not discrete frequency components unless the original signal is discrete in frequency.

5.1.2 Nyquist condition

$$F_s > 2F_{\text{max}}$$

This prevents overlap for a bandlimited lowpass signal with no content above F_{max} . If the signal may contain higher-frequency components, an anti-aliasing filter is needed.

5.1.3 Alias frequency for a sinusoid

$$f_{\text{alias}} = |f - kF_s| \quad \text{chosen in } 0 \leq f_{\text{alias}} \leq F_s/2$$

A 4 Hz tone sampled at 6 Hz aliases to 2 Hz. Use Hz for this folding formula; use rad/s only after multiplying by 2π .

Python aid: Use `scripts/sampling/adc.py` function `alias_frequency` when the problem asks for the observed alias of a single sinusoidal frequency. Inputs are signal frequency and sample rate in the same units. The script returns the folded alias frequency. Check manually that the task is a single-tone alias problem and not a full spectrum reconstruction question.

5.2 ADC quantization, LSB, kvantisering, dynamic range

5.2.1 Ideal ADC LSB size

$$\Delta V_{\text{LSB}} = \frac{V_{\text{max}} - V_{\text{min}}}{2^N}$$

For an ideal N -bit ADC, one LSB is the voltage range divided by the number of codes. Some conventions use $2^N - 1$ for endpoint spacing; use the convention implied by the exam statement.

Python aid: Use `scripts/sampling/adc.py` function `adc_lsb` when the problem asks for ideal ADC resolution from bit count and voltage range. Inputs are bits, V_{min} , and V_{max} . The script returns ΔV_{LSB} . Check manually whether the problem uses a 2^N or $2^N - 1$ convention.

5.2.2 Quantization noise and rounding

$$e_q \sim U \left[-\frac{\Delta V}{2}, \frac{\Delta V}{2} \right] \quad \text{for rounding to nearest}$$
$$\sigma_q^2 = \frac{\Delta V^2}{12}$$

Always-round-down quantization has biased error over a one-sided interval. Averaging reduces zero-mean random quantization noise more effectively than biased quantization error.

5.2.3 Ideal dynamic range

$$DR_{\text{ideal}} \approx 6.02N \text{ dB}$$

This is a fast bit-to-dB rule. Six bits correspond to about 36 dB and twelve bits to about 72 dB.

5.3 Anti-alias filter sizing, ADC and Butterworth

5.3.1 High-frequency Butterworth asymptote

$$|H_{\text{LP}}(f)| \approx \frac{1}{(f/f_{3dB})^n} \quad f \gg f_{3dB}$$

At the Nyquist frequency $F_s/2$, unwanted high-frequency content should be attenuated enough to avoid visible aliasing or to fall below one LSB, depending on the requirement.

5.3.2 Sampling rate for LSB-level attenuation

$$A_{\text{max}} \left(\frac{f_{3dB}}{F_s/2} \right)^n < \Delta V_{\text{LSB}}$$
$$F_s > 2f_{3dB} \left(\frac{A_{\text{max}}}{\Delta V_{\text{LSB}}} \right)^{1/n}$$

Use this when the exam states that the filtered amplitude at half the sampling frequency must be too small to toggle the LSB.

Python aid: Use `scripts/sampling/adc.py` function `required_sampling_rate_for_lsb` when the problem combines ADC resolution with a Butterworth anti-aliasing filter and asks for a minimum sampling rate. Inputs are bit count, voltage range, filter order, f_{3dB} , and worst-case amplitude if not equal to the range. The script returns minimum F_s . Check manually that the high-frequency asymptote is allowed and that f_{3dB} is in Hz.

5.4 Instrumentation amplifier, differential gain, common mode, CMRR

5.4.1 Differential gain

$$G_d = 1 + \frac{2R_2}{R_G}$$

This is the ideal gain formula for the course's three-op-amp instrumentation-amplifier model under matched-resistor assumptions.

5.4.2 Common-mode gain from mismatch

$$G_c = \frac{1 - \alpha}{2}$$

This simplified expression applies to the mismatch model where $R_{4b} = \alpha R_{4a}$. It should not be used for arbitrary simultaneous resistor mismatches.

5.4.3 Common-mode rejection ratio

$$CMRR = \left| \frac{G_d}{G_c} \right|, \quad CMRR_{dB} = 20 \log_{10}(CMRR)$$

If $G_c = 0$, the ideal CMRR is infinite. In exam statements, compare common-mode output error to the differential output signal in dB.

Python aid: Use `scripts/circuits/instrumentation.py` function `instrumentation_gains` when the problem gives R_2, R_G, α and asks for G_d, G_c , or CMRR. The script returns a dictionary with these gains. Check manually that the mismatch matches the single- α model and that the op-amp assumptions are ideal.

5.5 Exam recipe: ADC and instrumentation MCQ checks

5.5.1 Sampling and ADC checklist

Use when: an option mixes sampling, aliasing, ADC resolution, quantization noise, and anti-alias filter claims.

Steps:

1. Convert between f and ω only when needed.
2. Check Nyquist using the highest signal frequency after filtering.
3. Compute ΔV_{LSB} before comparing amplitudes.
4. Distinguish rounding-to-nearest from always-round-down quantization.

Fast checks: N bits gives about $6.02N$ dB; a 4th-order high-frequency asymptote gives -24 dB/octave.

6 Validated Python aids, Python-funktioner, kode

6.1 How to use scripts during the exam

6.1.1 Script use rule

Use scripts only after the formula, assumptions, and task type are identified from the notes. Scripts are calculation aids; they do not decide whether a conceptual statement is true.

6.1.2 Validated script lookup

Second-order poles: `scripts/lti/second_order.py`, `classify_second_order`. Check denominator $s^2 + a_1s + a_0$.

Step features from parameters: `scripts/lti/second_order.py`, `step_features_from_zeta_wn`. Check $0 < \zeta < 1$.

Step plot estimates: `scripts/lti/second_order.py`, `estimate_from_overshoot_peak_time`. Check underdamped second-order plot.

Impulse, step, and ramp response: `scripts/lti/responses.py`, `impulse_response_from_transfer`, `step_response_from_transfer`, `ramp_response_from_transfer`, `related_unit_responses`. Check zero-state transfer function and causal LTIC assumption.

ODE to transfer function: `scripts/lti/responses.py`, `transfer_from_ode`. Check zero initial conditions.

Zero-input Laplace: `scripts/lti/responses.py`, `zero_input_laplace_second_order`. Check normalized second-order ODE.

Convolution: `scripts/transforms/convolution.py`, `convolve_causal`. Check both signals are causal.

Inverse Laplace: `scripts/transforms/laplace.py`, `inverse_laplace_rational`. Check rational coefficients.

Fourier coefficients: `scripts/fourier/series.py`, `complex_fourier_coefficients`. Check one full period.

Fourier property: `scripts/fourier/properties.py`, `exponential_transform_variant`. Check scaling or modulation only.

Bode and pole-zero: `scripts/frequency/bode.py`, `bode_values`, `classify_filter_limits`, `transfer_from_poles_zeros`. Check rad/s and conjugate pairs.

Butterworth and RC scaling: `scripts/filters/butterworth.py`, `butterworth_lowpass_order`, `butterworth_highpass_order`, `butterworth_poles_q`, `frequency_scale_rc`. Check passband/stopband order and rad/s conversion.

ADC and sampling: `scripts/sampling/adc.py`, `adc_lsb`, `alias_frequency`, `required_sampling_rate_for_lsb`. Check ADC convention and single-tone/asymptote assumptions.

Instrumentation amplifier: `scripts/circuits/instrumentation.py`, `instrumentation_gains`. Check single- α mismatch model.